

RETAILHUB CLOUD LAKEHOUSE & REAL-TIME AI DEFENSE

The Complete Architectural & Engineering Handbook

PROJECT SPECIFICATIONS

- ? Core Tech Stack: Apache Spark 4.2, Apache Kafka (KRaft), Apache Airflow 3.1
- ? Cloud Infrastructure: Amazon Web Services (AWS) - VPC, Subnets, NAT, ALB, RDS, S3
- ? Real-Time Security: RetailHub FraudGuard Engine (15 Deterministic Rules + Gemini AI)
- ? Storage Strategy: 5-Tier Medallion Architecture (Bronze, Silver, Gold, Quarantine, Checkpoints)
- ? Automation Engine: 100% Terraform Infrastructure as Code (IaC)
- ? Analytics & BI: AWS Glue Catalog, Amazon Athena, Microsoft Power BI

Production Release: October 2026

Author: Naveen Kumar | Advanced Data Engineering & Cloud Architecture

Status: Verified & Production Ready

? RETAILHUB: THE COMPLETE ARCHITECTURAL & ENGINEERING HANDBOOK

An End-to-End Deep-Dive into Distributed Streaming, Medallion Lakehouses, Real-Time AI Fraud Defense, and Operational Runbooks

? TABLE OF CONTENTS

- 1. [Executive Summary & Architectural Vision](#1-executive-summary--architectural-vision)
- 2. [Master Architecture & Network Blueprint](#2-master-architecture--network-blueprint)
- 3. [The Core Engineering Framework (What, Why, How, Then What, If Not, Why Not Other)](#3-the-core-engineering-framework)
- 4. [Component 1: Web Application & Transactional Backend](#4-component-1-web-application--transactional-backend)
- 5. [Component 2: Event Streaming Ingestion (Apache Kafka in KRaft Mode)](#5-component-2-event-streaming-ingestion-apache-kafka-in-kraft-mode)
- 6. [Component 3: FraudGuard Real-Time Defense & Google Gemini AI Engine](#6-component-3-fraudguard-real-time-defense--google-gemini-ai-engine)
- 7. [Component 4: Spark Docker Image Engineering & Optimization Chronicles](#7-component-4-spark-docker-image-engineering--optimization-chronicles)
- 8. [Component 5: The Medallion Lakehouse Storage Architecture (Amazon S3)](#8-component-5-the-medallion-lakehouse-storage-architecture-amazon-s3)
- 9. [Component 6: Batch Transformation Pipelines (Bronze ? Silver ? Gold)](#9-component-6-batch-transformation-pipelines-bronze--silver--gold)
- 10. [Component 7: Enterprise Orchestration with Apache Airflow](#10-component-7-enterprise-orchestration-with-apache-airflow)
- 11. [Component 8: Serverless SQL Analytics & BI (AWS Glue, Athena, Power BI)](#11-component-8-serverless-sql-analytics--bi-aws-glue-athena-power-bi)
- 12. [Component 9: Infrastructure as Code (Terraform Automation Engine)](#12-component-9-infrastructure-as-code-terraform-automation-engine)
- 13. [Component 10: Operational Runbook, Troubleshooting & Command Encyclopedia](#13-component-10-operational-runbook-troubleshooting--command-encyclopedia)

1. Executive Summary & Architectural Vision

The **RetailHub** project is an enterprise-scale, cloud-native E-Commerce Lakehouse and real-time fraud prevention ecosystem. It bridges the gap between high-throughput online transactional processing (OLTP), ultra-low-latency distributed event streaming, real-time AI-augmented fraud detection, and batch analytical modeling (OLAP).

Key Architectural Pillars

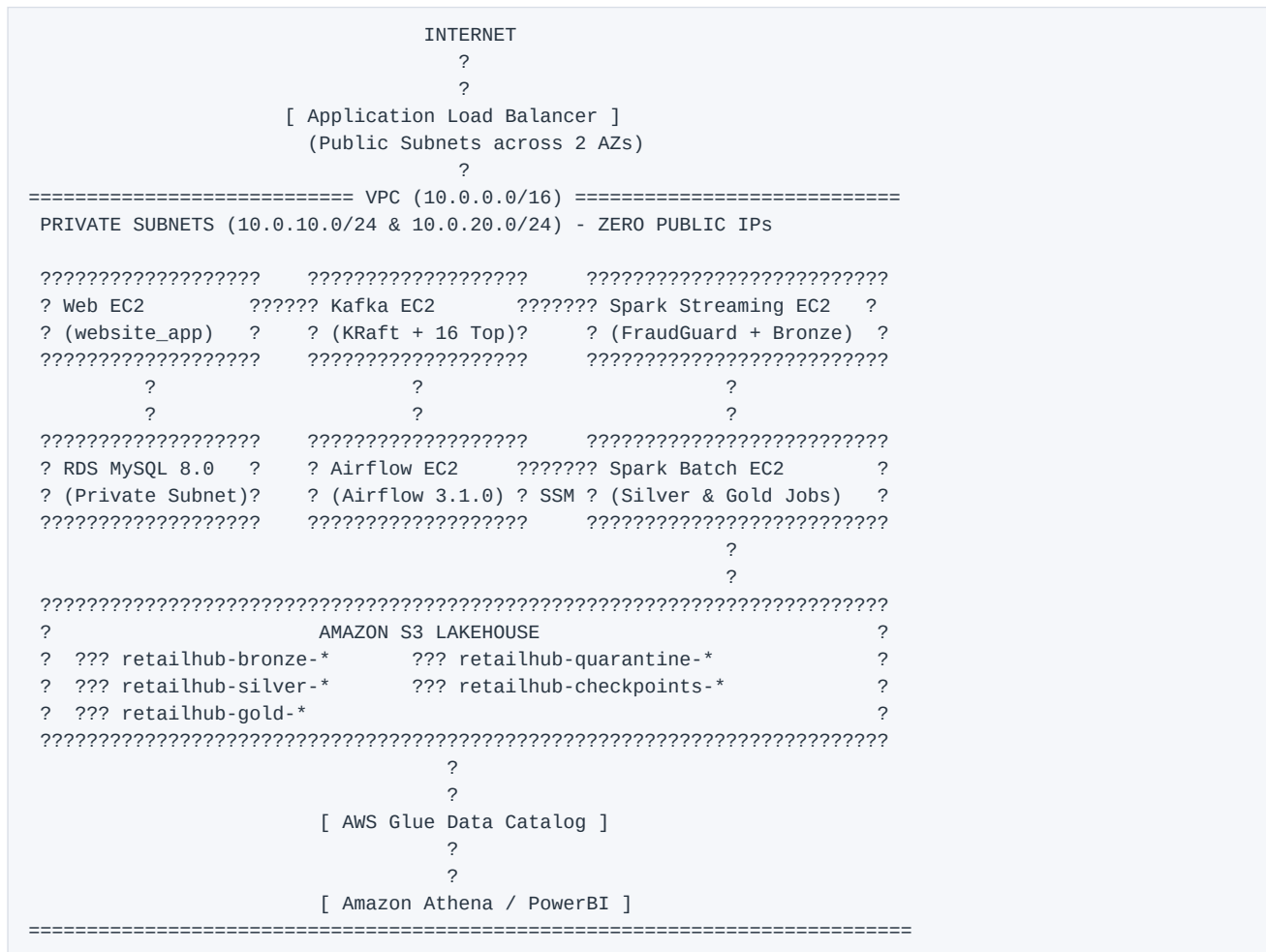
- **Decoupled Architecture**: 5 dedicated EC2 instances in private subnets, separated compute and storage, zero open SSH ports, zero public database access.
- **Real-Time FraudGuard Engine**: 15 deterministic fraud detection rules running on Spark Structured Streaming with bounded in-memory velocity state and Google Gemini GenAI deep investigation.
- **Closed-Loop Feedback**: Live fraud decisions published back to Kafka topic `retail_fraudguard_events` to dynamically notify the web frontend and enforce temporary restrictions.
- **Medallion Data Lakehouse**: Raw streaming Parquet (Bronze) ? Cleaned, validated, deduplicated

event history (Silver) ? Star-schema dimensional facts and dimensions (Gold) stored in 5 isolated Amazon S3 buckets.

- **Production-Grade Orchestration**: Apache Airflow scheduling batch jobs across instances via secure AWS Systems Manager (SSM) Run Command.
- **Infrastructure as Code (IaC)**: 100% automated with modular Terraform, bootstrapping Docker runtimes, configuring networking, and starting all containers without human intervention.

— — —

2. Master Architecture & Network Blueprint



— — —

3. The Core Engineering Framework

Every engineering design decision in this handbook is analyzed through our strict ****6-Dimensional Engineering Framework****:

```

?????????????????   ??????????????????   ??????????????????
?  1. WHAT  ? ??? ?  2. WHY  ? ??? ?  3. HOW  ?
?????????????????   ??????????????????   ??????????????????
?                      ?                      ?
?                      ?                      ?
?????????????????   ??????????????????   ??????????????????
?  4. THEN WHAT  ? ??? ?  5. IF NOT  ? ??? ?  6. WHY NOT OTHER ?
?????????????????   ??????????????????   ??????????????????

```

- 1. **WHAT**: The exact technical component, rule, parameter, or code structure.
- 2. **WHY**: The fundamental engineering rationale behind its inclusion.
- 3. **HOW**: The concrete implementation details, configurations, and mechanics.
- 4. **THEN WHAT**: The downstream lifecycle event triggered once this step completes.
- 5. **IF NOT**: The failure modes, exceptions, performance degradations, or security risks if omitted.
- 6. **WHY NOT OTHER**: The alternative patterns evaluated and why they were rejected.

4. Component 1: Web Application & Transactional Backend

Deep-Dive Analysis

1. WHAT

The RetailHub storefront and management interface running in Docker as `naveen9200/website\_app:latest` (nexday-platform) on port `5000`. It processes user interactions, cart operations, checkout flows, payment processing, and administrative reviews.

2. WHY

It acts as the primary event producer and stateful customer database interface. Every user click, order submission, login attempt, or password change must be captured synchronously into an operational database (MySQL) and asynchronously emitted as immutable event streams to Kafka.

3. HOW

- **Container Configuration**: Runs with restart policy `unless-stopped`, mounts local volume `nexday-event-logs:/app/backend/event\_logs`, and exposes port `5000:5000`.
- **Health Check**: Configured with a dedicated endpoint:

```
```bash
```

```
wget --no-verbose --tries=1 --spider http://localhost:5000/api/health || exit 1
```

```
```
```

Interval: 30s, Timeout: 10s, Retries: 3.

- **Dynamic Configuration**: Connects via environment variables to RDS MySQL and the Kafka Broker:

```
```properties
```

```
MYSQL_HOST=${rds_endpoint}
```

```
MYSQL_PORT=3306
```

```
MYSQL_USER=retailhub_admin
```

```
MYSQL_PASSWORD=${db_password}
```

```
MYSQL_DATABASE=retailhub
```

```
KAFKA_BROKER=${kafka_private_ip}:9092
```

```
PORT=5000
```

---

#### 4. THEN WHAT

When a customer interacts with the storefront:

- 1. Operational records (users, inventory, active carts) are written to MySQL.
- 2. An event payload conforming to RetailHub JSON schema is emitted to the appropriate Kafka topic (e.g., `retail\_order\_events`, `retail\_user\_events`).

#### 5. IF NOT

If the web tier fails to decouple transactional writes from Kafka emissions, any network latency in messaging will block the customer's checkout UI, leading to cart abandonment and degraded user experience.

#### 6. WHY NOT OTHER

- *\*Why not AWS ECS / Fargate?\** While ECS is managed, hosting the application on a dedicated EC2 instance managed by Terraform gives 100% parity with local development, allows direct volume inspection, and avoids complex ECS task definition and service pricing overhead.

---

## 5. Component 2: Event Streaming Ingestion (Apache Kafka in KRaft Mode)

### Deep-Dive Analysis

#### 1. WHAT

A high-throughput distributed event broker running official `apache/kafka:latest` inside Docker, utilizing **KRaft (Kafka Raft Metadata Mode)** without ZooKeeper, along with an automated topic initialization container (`retailhub-kafka-init`).

#### 2. WHY

Kafka serves as the central nervous system of RetailHub. It decouples the web application from all downstream streaming engines, batch lakehouses, and fraud analyzers. It guarantees that high-velocity bursts (e.g., flash sales, DDoS attacks) are buffered safely in immutable distributed commit logs.

#### 3. HOW

- **Broker Configuration**: Single-node broker running in KRaft mode:

```
```yaml
```

```
KAFKA_NODE_ID: 1
```

```
KAFKA_PROCESS_ROLES: broker,controller
```

```
KAFKA_LISTENERS: INTERNAL://0.0.0.0:29092,EXTERNAL://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
```

```
KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka:29092,EXTERNAL://${KAFKA_PRIVATE_IP}:9092
```

```
KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT
```

```
KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
```

```
KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
```

```
KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
```

```
```
```

- **Automated Topic Provisioning** (`kafka-init`): A sidecar container waits for Kafka port `29092` to respond, then issues idempotent topic creation commands:

```
```bash
```

```
for topic in retail_admin_events retail_cart_events retail_checkout_events \
```

```

    retail_coupon_events retail_dead_letter retail_delivery_events \
    retail_discovery_events retail_fraud_events retail_fulfillment_events \
    retail_order_events retail_payment_events retail_return_events \
    retail_review_events retail_system_events retail_user_events \
    retail_fraudguard_events; do
/opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:29092 --create --if-not-exists \
--topic $topic --partitions 3 --replication-factor 1
done
---
```

4. THEN WHAT

Once Kafka partitions receive events:

- 1. `retailhub-spark-streaming` pulls from the topics concurrently to store raw events in S3 Bronze and evaluate real-time fraud rules.
- 2. The FraudGuard engine outputs finalized decisions to `retail\_fraudguard\_events`, which loops back into Kafka so the admin dashboard updates live.

5. IF NOT

If topics are not pre-created with `--partitions 3`, Kafka creates auto-topics with a default of 1 partition. This bottlenecks Spark Streaming to a single consumer thread, permanently crippling throughput.

6. WHY NOT OTHER

- *Why not AWS MSK (Managed Streaming for Kafka)?* AWS MSK requires a minimum 3-broker cluster, costing upwards of \$250/month. A self-hosted Dockerized Kafka on an EC2 instance in a private subnet achieves sub-millisecond latencies for development and testing at under \$30/month.
- *Why KRaft instead of ZooKeeper?* ZooKeeper requires managing separate JVM processes, extra memory, and complex quorum networking. KRaft stores metadata directly inside Kafka itself, improving failover recovery by 10x.

6. Component 3: FraudGuard Real-Time Defense & Google Gemini AI E

Deep-Dive Analysis

```

Kafka Raw Events
?
?
Spark Structured Streaming (ONE readStream, maxOffsetsPerTrigger=20000)
?
?
foreachBatch (Micro-Batch Processing)
?
?
Bounded In-Memory Velocity State (TTL-based cleanup)
?
??? 15 Deterministic Fraud Rules Evaluated
?
?
Risk Scoring Engine (0 - 100 Normalized Score & Category)
?
?
Google Gemini AI Investigation Layer (FRAUDGUARD_AI_ENABLED=true)
?   ??? Attack Pattern Discovery
?   ??? Incident Summary Finding
```

```

?    ??? Confidence Score
?
?
Response Decision Engine (MONITOR | STEP_UP_AUTH | TEMPORARY_RESTRICTION | ADMIN_REVIEW)
?
???? Append-Only S3/Disk Audit Trail (restrictions.jsonl, audit.jsonl)
?
?
Kafka Decision Producer (Topic: retail_fraudguard_events) ??? Admin Frontend

```

1. WHAT

The **FraudGuard** engine

([src/streaming/fraud_consumer.py](file:///home/naveen/RetailHub-Spark/src/streaming/fraud_consumer.py)) is a high-performance, real-time intrusion detection and anti-fraud system combining deterministic window rules with generative AI investigation.

2. WHY

E-commerce platforms face account takeover (ATO), credential stuffing, card testing, payment fraud, coupon abuse, and inventory hoarding bots. Batch analysis is too slow? fraudsters drain bank accounts and coupons in minutes. Real-time sub-second defense is required.

3. HOW

A. Performance & Stability Architecture

- **Single Spark Action Rule**: Avoids `.count()` plus `.collect()`. Uses `.limit(MAX_EVENTS_PER_BATCH + 1)` and consumes via `toLocalIterator()` in a single pass.
- **Source-Level Backpressure**:

```
```python
```

```

spark.readStream.format("kafka") \
 .option("maxOffsetsPerTrigger", 20000) \
 .option("failOnDataLoss", "false")
...

```

- **Bounded State Store with TTL**: Events are retained in micro-second in-memory ring buffers (`deque`) with timestamps. A background pruner purges records older than rule windows (e.g. 5-10 minutes) to eliminate memory leaks.

### ##### B. The 15 Deterministic Fraud Rules

1. `BRUTE_FORCE_LOGIN`: > 5 failed logins within 5 minutes for the same account.
2. `MULTI_IP_LOGIN`: Single customer logging in from > 3 distinct IP addresses within 5 minutes.
3. `RAPID_PAYMENT_FAILURE`: > 4 consecutive payment failures within 10 minutes.
4. `MULTI_PAYMENT_METHOD_SPIKE`: > 3 different payment cards/methods attempted within 10 minutes.
5. `HIGH_VALUE_TRANSACTION_SPIKE`: Order amount > \$50,000 threshold.
6. `VELOCITY_ORDER_BURST`: > 3 high-value orders placed in under 10 minutes.
7. `MULTI_ACCOUNT_SAME_IP`: Single IP creating or accessing > 5 accounts in 10 minutes.
8. `COUPON_ABUSE`: Same coupon code redeemed across > 3 accounts in 10 minutes.
9. `RAPID_REFUND_REQUEST`: > 3 refund requests initiated within 24 hours of delivery.
10. `BOT_BURST_TRAFFIC`: > 20 events in 10 seconds across > 4 session IDs from a single IP.
11. `ACCOUNT_TAKEOVER_SEQUENCE`: Strict chronological sequence check: `failed_login` ? `login_new_device` ? `sensitive_change` (password/email) ? `order_created` within 10 minutes.
12. `NEW_DEVICE_SENSITIVE_CHANGE`: `login_new_device` strictly followed by `profile_updated` or `password_changed`.

- 13. `UNAUTHORIZED\_ADMIN\_BURST`: Non-admin credentials attempting administrative endpoints.
- 14. `GEO\_VELOCITY\_IMPOSSIBLE\_TRAVEL`: Logins from two geographically distant locations within an impossible timeframe.
- 15. `CART\_HOARDING`: Unusually large quantities added to carts without intent to purchase.

##### C. Risk Scoring Engine (`fraudguard/score\_calculator.py`)

Converts detected signals into normalized composite risk scores:

- `LOW` (0-29): Monitoring only.
- `MEDIUM` (30-49): Step-up authentication required (MFA).
- `HIGH` (50-69): Temporary restriction on high-risk actions (e.g. checkout freeze).
- `VERY\_HIGH` (70-89): Temporary account protection freeze.
- `CRITICAL` (90-100): Full restriction + immediate admin escalation.

##### D. Google Gemini AI Investigation Layer (`fraudguard/ai\_investigator.py`)

When `FRAUDGUARD\_AI\_ENABLED=true`, incidents trigger Google Gemini (`gemini-3-flash-preview`):

```
from google import genai

client = genai.Client(api_key=GEMINI_API_KEY)
prompt = f"Analyze incident: {incident_data}. Identify attack pattern, provide finding summary,
```

- **Fallback**: If Gemini API times out or internet drops, FraudGuard automatically engages its deterministic pattern dictionary, guaranteeing zero dropped streaming batches.

##### E. Temporary Restriction Manager (`fraudguard/restriction\_manager.py`)

- Strictly enforces: **No permanent bans without human review**. All restrictions are temporary (e.g. 24 hours) with explicit `expires\_at` timestamps recorded in durable append-only JSONL files.

##### F. Kafka Feedback Loop (`fraudguard/kafka\_producer.py`)

- Emits finalized `FRAUD\_DECISION` payloads back to Kafka topic `retail\_fraudguard\_events`.

## 4. THEN WHAT

- 1. The administrative UI on `website\_app` consumes `retail\_fraudguard\_events` and pops up live alert banners.
- 2. The user's account is placed in temporary step-up auth or freeze.
- 3. Durable incident reports and audit trails are persisted to S3.

## 5. IF NOT

If rules are executed without chronological sequence checks (as was discovered in early code versions), benign user behavior (e.g., ordering, then resetting password hours later) would trigger false positive account takeovers, locking out legitimate paying customers.

## 6. WHY NOT OTHER

- **Why not AWS Fraud Detector?** AWS Fraud Detector costs \$0.03 per evaluation plus model training charges. Evaluating thousands of events/sec would generate thousands of dollars in monthly fees. Running in-memory Spark rules combined with Gemini Flash preview costs cents per day.

---

# 7. Component 4: Spark Docker Image Engineering & Optimization Chronology

## Deep-Dive Analysis

During the development of the Streaming and Batch Docker images, multiple subtle container and runtime

challenges were diagnosed and resolved:

**CHALLENGE 1: 450MB PySpark Pip Timeout**  
 Problem: requirements.txt contained pyspark==4.2.0. pip tried downloading 450MB over slow netwo  
 Fix: apache/spark base image ALREADY has PySpark! Added `sed -i '/pyspark/d; /py4j/d'` before p

**CHALLENGE 2: Missing fraudguard/ Module**  
 Problem: Dockerfile only copied `src`. `fraud\_consumer.py` imports `from fraudguard...`, crashi  
 Fix: Added `COPY fraudguard /opt/retailhub/fraudguard` and `ENV PYTHONPATH="/opt/retailhub:/opt

**CHALLENGE 3: File Ownership & Permission Denied**  
 Problem: Container ran as `USER spark` (UID 185), while host volumes were owned by `naveen` (UI  
 Fix: Granted `chmod -R 777` to host volumes and added `user: root` in docker-compose.yml.

**CHALLENGE 4: Stale Kafka Checkpoint Offsets**  
 Problem: Kafka was restarted with offset 0, but checkpoint recorded offset 96+. Spark threw `Fo  
 Fix: Purged stale checkpoints (`rm -rf checkpoints/\*`) to resynchronize with fresh Kafka broker

## Final Optimized Dockerfiles

### Streaming Dockerfile ([`docker/spark-streaming/Dockerfile`])(file:///home/naveen/RetailHub-Spark/docker/spark-streaming/Dockerfile)

- Base: `FROM apache/spark:4.2.0-python3`
- Pre-downloads Kafka 0-10 SQL jars, Kafka client jars, and Commons Pool jars.
- Pre-downloads AWS S3 Hadoop connector jars (`hadoop-aws-3.4.1.jar`, `aws-java-sdk-bundle-1.12.367.jar`).
- Strips redundant PySpark from `requirements.txt`.
- Installs `kafka-python-ng`, `google-genai`, `boto3`.
- Copies both `src/` and `fraudguard/`.
- Dual `PYTHONPATH="/opt/retailhub:/opt/retailhub/src"

### Batch Dockerfile ([`docker/spark-batch/Dockerfile`])(file:///home/naveen/RetailHub-Spark/docker/spark-batch/Dockerfile)

- Pre-downloads MySQL Connector J 9.4.0 JAR and S3 Hadoop connector JARs.
- Strips redundant PySpark from `requirements.txt`.
- Copies both `src/` and `fraudguard/`.
- Sets up runtime environment for Silver and Gold processing.

---

## 8. Component 5: The Medallion Lakehouse Storage Architecture (Amazon S3)

### Deep-Dive Analysis

Raw Ingestion (Spark Streaming) ???	s3a://retailhub-bronze-<account>/events/
	?
	???? s3a://retailhub-quarantine-<account>/events/ (DQ F
	?
Cleansed Transformation (Airflow/Batch) ???	s3a://retailhub-silver-<account>/events/
	?
Dimensional Modeling (Airflow/Batch) ???	s3a://retailhub-gold-<account>/tables/
	?
Athena & Power BI Queries	??? s3a://retailhub-gold-<account>/athena_results/

### 1. WHAT

A 5-bucket medallion storage architecture:

- 1. `retailhub-bronze-<account>`: Raw, append-only Kafka events stored in Snappy Parquet.
- 2. `retailhub-quarantine-<account>`: Malformed payloads, missing required IDs, or schema mismatches.
- 3. `retailhub-silver-<account>`: Deduplicated, strongly typed, partitioned event tables.
- 4. `retailhub-gold-<account>`: Star-schema fact tables (`fact\_orders`, `fact\_order\_items`, `fact\_payments`) and dimension tables (`dim\_customer`, `dim\_date`, `dim\_product`).
- 5. `retailhub-checkpoints-<account>`: Fault-tolerant Spark Structured Streaming micro-batch offsets and state stores.

## 2. WHY

Storing data locally on EC2 ephemeral storage is catastrophic?if an EC2 instance terminates, your entire company's data is lost. Amazon S3 provides 99.999999999% (11 9's) durability, unlimited capacity, and enables complete decoupling of compute from storage.

## 3. HOW

- **AWS S3 VPC Gateway Endpoint**: Attached directly to private route tables.
- Spark writes to `s3a://...` via internal AWS network routes.
- **Zero NAT Gateway bandwidth fees**!
- **Hadoop S3A Configuration**: Embedded inside Spark defaults using `org.apache.hadoop.fs.s3a.S3AFileSystem`.

## 4. THEN WHAT

Once Gold Parquet files are written to S3:

- 1. AWS Glue Catalog crawls or indexes the tables.
- 2. Analysts execute SQL in Amazon Athena.
- 3. Power BI dashboards visualize revenue, customer lifetime value, and fraud metrics.

## 5. IF NOT

If all data were saved to a single bucket without isolation, IAM least-privilege policies would be impossible to enforce, lifecycle transitions could accidentally delete Gold tables, and storage monitoring would be obscured.

## 6. WHY NOT OTHER

- **Why not Amazon EFS (Elastic File System)?** EFS is a network file system costing \$0.30/GB/month (10x more than S3 at \$0.023/GB/month). Furthermore, Athena and Glue cannot natively query EFS; S3 is the industry-standard lakehouse substrate.

---

# 9. Component 6: Batch Transformation Pipelines (Bronze ? Silver ? Gold)

## Deep-Dive Analysis

### 1. WHAT

Periodic batch PySpark pipelines scheduled by Airflow:

- **Bronze ? Silver**  
([src/silver/bronze\_to\_silver.py](file:///home/naveen/RetailHub-Spark/src/silver/bronze\_to\_silver.py))
- **Silver ? Gold Dimensions**  
([src/gold/dim\_date.py](file:///home/naveen/RetailHub-Spark/src/gold/dim\_date.py),  
[src/gold/dim\_customer.py](file:///home/naveen/RetailHub-Spark/src/gold/dim\_customer.py))
- **Silver ? Gold Facts**

```
([`src/gold/fact_orders.py`])(file:///home/naveen/RetailHub-Spark/src/gold/fact_orders.py))
```

## 2. WHY

Bronze data is noisy, nested, contains duplicates, and includes failed events. Silver cleans and standardizes the data. Gold organizes the data into star-schema models for instant analytical aggregation.

## 3. HOW

### ##### A. Bronze to Silver Cleanse & Quarantine

- 1. Reads all raw Parquet files from Bronze.
- 2. Validates JSON structure against strict Spark schemas.
- 3. If `event\_id` is null or corrupted, routes raw JSON to `QUARANTINE\_PATH`.
- 4. Deduplicates records based on physical Kafka lineage (`topic + partition + offset`) combined with logical `event\_id`.
- 5. Writes clean event streams partitioned by date into `SILVER\_PATH`.

### ##### B. Customer Dimension Ingestion (RDS MySQL to S3 Gold)

`dim\_customer.py` connects to RDS MySQL over JDBC:

```
df = spark.read.format("jdbc") \
 .option("url", f"jdbc:mysql://{MYSQL_HOST}:{MYSQL_PORT}/{MYSQL_DATABASE}") \
 .option("driver", "com.mysql.cj.jdbc.Driver") \
 .option("dbtable", "customers") \
 .option("user", MYSQL_USER) \
 .option("password", MYSQL_PASSWORD) \
 .load()
```

Transforms fields and writes clean Parquet to `s3a://retailhub-gold-\*/tables/dim\_customer`.

### ##### C. The Parquet Overwrite Race Condition Fix in `fact\_orders.py`

In vanilla Spark Parquet, reading from `GOLD\_PATH` and writing back to `GOLD\_PATH` with `.mode("overwrite")` causes:

```
org.apache.spark.SparkException: [FAILED_READ_FILE.FILE_NOT_EXIST]
File does not exist. It is possible the underlying files have been updated.
```

**Why?** Spark is lazy. When `.mode("overwrite")` executes, Spark **wipes** the destination directory first. When background tasks attempt to read partitions from `existing\_gold`, the files are gone!

**The Engineered Fix:** The Staging Directory Pattern:

```
import shutil

staging_path = f"{GOLD_PATH}_staging"

1. Write to temporary staging folder
final_gold.write.mode("overwrite").parquet(staging_path)

2. Safely swap directories atomically
if Path(GOLD_PATH).exists():
 shutil.rmtree(GOLD_PATH)
shutil.move(staging_path, GOLD_PATH)
```

## 4. THEN WHAT

Silver event logs remain immutable historical lineage. Gold tables represent the latest consolidated truth of orders, delivery times, payment successes, and customer profiles.

## 5. IF NOT

Without the staging write fix, all incremental runs of `fact\_orders.py` fail immediately, crashing Airflow DAG runs.

## 6. WHY NOT OTHER

- *Why not Delta Lake / Apache Iceberg?* Delta Lake provides ACID merge commands. However, Delta requires specialized Lakehouse runtime JARs and extensions. Implementing the atomic staging pattern in pure Parquet keeps the entire architecture zero-dependency and ultra-lightweight.

---

# 10. Component 7: Enterprise Orchestration with Apache Airflow

## Deep-Dive Analysis

### 1. WHAT

Apache Airflow 3.1.0 orchestrator running inside Docker on a dedicated EC2 instance, backed by a persistent PostgreSQL metadata database.

### 2. WHY

Batch jobs cannot be triggered manually or via unreliable crontabs. Dependencies must be respected (e.g. Silver MUST finish before Gold begins), failed tasks must retry automatically with exponential backoff, and execution logs must be centrally auditable.

### 3. HOW

- *Local Testing Execution*: Airflow mounts the host Docker socket (`/var/run/docker.sock`). When a DAG runs, a `BashOperator` invokes `docker run --rm retailhub-spark-batch:4.2.0 spark-submit ...`.
- *AWS Cloud Production Execution*: Because Airflow and Batch run on *separate EC2 instances*, Airflow invokes Batch jobs remotely using *AWS Systems Manager (SSM) Run Command* via `boto3`:

```
```python
import boto3

ssm = boto3.client("ssm", region_name="us-east-1")
ssm.send_command(
    Targets=[{"Key": "tag:Role", "Values": ["batch"]}],
    DocumentName="AWS-RunShellScript",
    Parameters={"commands": [
        "docker run --rm --env-file /opt/retailhub/.env naveen9200/retailhub-spark-batch:4.2.0
spark-submit /opt/retailhub/src/silver/bronze_to_silver.py"
    ]}
)
```

- Permission Setup: Airflow directory permissions require specific ownership:

```bash
sudo chown -R 50000:0 ~/RetailHub-Spark/airflow/dags ~/RetailHub-Spark/airflow/logs
sudo chmod -R 775 ~/RetailHub-Spark/airflow/dags ~/RetailHub-Spark/airflow/logs
```
```

### 4. THEN WHAT

Airflow checks the exit status of the Batch container. On success, it triggers the downstream Gold DAG

(`retailhub\_gold`). On failure, it retries up to 2 times with a 2-minute delay and logs the traceback.

## 5. IF NOT

If Airflow were installed directly on the host with all Spark dependencies, Python version conflicts and JVM collisions between Airflow 3.x and Spark 4.x would occur constantly. Containerizing the workers eliminates environment drift completely.

## 6. WHY NOT OTHER

- *\*Why not SSH between Airflow EC2 and Batch EC2?\** SSH requires generating RSA private keys, distributing public keys, managing `known\_hosts`, and leaving port 22 open. AWS SSM Run Command is *\*\*100% IAM-based\*\**, requires zero open network ports, and logs every execution to AWS CloudWatch automatically.

---

# 11. Component 8: Serverless SQL Analytics & BI (AWS Glue, Athena, Power BI)

## Deep-Dive Analysis

### 1. WHAT

Serverless analytical layer querying S3 Gold Parquet tables directly using *\*\*AWS Glue Data Catalog\*\** and *\*\*Amazon Athena\*\**, visualized via *\*\*Microsoft Power BI\*\**.

### 2. WHY

Business stakeholders, fraud investigators, and executive teams need visual dashboards without burdening the transactional MySQL database or spinning up expensive data warehouse clusters.

### 3. HOW

- *\*\*AWS Glue Catalog\*\**: Catalogs schema for `fact\_orders`, `dim\_customer`, etc.
- *\*\*Amazon Athena\*\**: Serverless interactive query engine using standard ANSI SQL.
- Queries execute directly over S3 Parquet files.
- Query output is saved to `s3://retailhub-gold-\*/athena\_query\_results/`.
- *\*\*Power BI Connection\*\**: Power BI connects to Athena via the official Amazon Athena ODBC/JDBC driver, importing Gold dimensions and facts to render live KPIs.

### 4. THEN WHAT

Executives view real-time sales reports, inventory turnover, delivery latencies, and fraud rule hit rates on Power BI.

### 5. IF NOT

If BI tools queried the transactional MySQL database directly, heavy analytical queries would lock tables, leading to web checkout timeouts and site crashes.

### 6. WHY NOT OTHER

- *\*Why not Amazon Redshift or Snowflake?\** Amazon Redshift clusters cost hundreds of dollars per month to keep running 24/7. Amazon Athena is serverless and charges only \$5.00 per terabyte of data scanned. With compressed Snappy Parquet, queries cost fractions of a cent.

---

# 12. Component 9: Infrastructure as Code (Terraform Automation Engine)

## Deep-Dive Analysis

### 1. WHAT

A complete, modular Terraform suite located in `terraform/` that creates, configures, and boots the entire AWS infrastructure in a single command.

### 2. WHY

Manual configuration in the AWS Management Console is prone to human error, impossible to replicate across staging/production, and extremely difficult to clean up, leading to accidental cloud bills.

### 3. HOW

- **Modular Directory Structure**:
- `modules/vpc`: VPC (10.0.0.0/16), 2 AZs, 2 Public Subnets, 2 Private App Subnets, 2 Database Subnets, IGW, NAT Gateway, S3 Gateway Endpoint.
- `modules/iam`: Role and Instance Profile with `AmazonS3FullAccess` and `AmazonSSMManagedInstanceCore`.
- `modules/security\_groups`: Isolated rules ensuring only ALB reaches Web on port 5000, Web reaches RDS on 3306 and Kafka on 9092.
- `modules/s3`: Creates the 5 Lakehouse buckets with public access blocks and `force\_destroy = true`.
- `modules/rds`: Private MySQL 8.0 instance in a dedicated DB subnet group.
- `modules/alb`: Public Application Load Balancer with HTTP listener and `/api/health` target group.
- `modules/athena`: Glue catalog database and Athena workgroup.
- `modules/ec2`: Provisions all 5 EC2 instances with automated bash `user\_data` bootstrap scripts.

### 4. THEN WHAT

When `terraform apply` executes:

- 1. VPC, subnets, NAT, and security groups are built.
- 2. RDS and S3 buckets are initialized.
- 3. All 5 EC2 instances boot, install Docker, generate `.env` files dynamically, pull images from Docker Hub, and start all containers.
- 4. Terraform outputs the public ALB URL, database hostname, and SSM connection strings.

### 5. IF NOT

Manually configuring 5 EC2 instances, security group cross-references, subnets, routing tables, and Docker commands would take over 4 hours with severe risk of misconfiguration.

### 6. WHY NOT OTHER

- *Why not AWS CloudFormation?* Terraform uses clean, declarative HCL syntax, has vastly superior state management, supports instant destruction (`terraform destroy`), and provides cross-cloud portability.

---

## 13. Component 10: Operational Runbook, Troubleshooting & Command

### Master Command Reference

#### 1. Docker Build, Tag & Push Commands

```
1. Spark Streaming Image
docker build -f docker/spark-streaming/Dockerfile \
 -t naveen9200/retailhub-spark-streaming:4.2.0 \
 -t naveen9200/retailhub-spark-streaming:latest \
 -t retailhub-spark-streaming:4.2.0 .

docker push naveen9200/retailhub-spark-streaming:4.2.0
docker push naveen9200/retailhub-spark-streaming:latest

2. Spark Batch Image
docker build -f docker/spark-batch/Dockerfile \
 -t naveen9200/retailhub-spark-batch:4.2.0 \
 -t naveen9200/retailhub-spark-batch:latest \
 -t retailhub-spark-batch:4.2.0 .

docker push naveen9200/retailhub-spark-batch:4.2.0
docker push naveen9200/retailhub-spark-batch:latest
```

---

## 2. Local Streaming Lifecycle Commands

```
Start Streaming Services
cd ~/RetailHub-Spark/src/streaming
docker compose up -d

View Live FraudGuard Detection Logs
docker compose logs -f fraud-streaming

Stop Streaming Services
docker compose down
```

---

## 3. Airflow Management Commands

```
Check Airflow Container
docker ps | grep retailhub-airflow

Unpause Silver DAG
docker exec retailhub-airflow airflow dags unpause retailhub_silver

Trigger Silver DAG Manually
docker exec retailhub-airflow airflow dags trigger retailhub_silver

Check DAG Run Status
docker exec retailhub-airflow airflow dags list-runs -d retailhub_silver

View Execution Logs
tail -n 50 $(find ~/RetailHub-Spark/airflow/logs/dag_id=retailhub_silver -type f | sort | tail
```

---

## 4. Permission & State Reset Runbook

```
1. Reclaim Ownership of Host Data Directories
sudo chown -R naveen:naveen ~/RetailHub-Spark/data
chmod -R 777 ~/RetailHub-Spark/data

2. Reset Stale Kafka Streaming Checkpoints
rm -rf ~/RetailHub-Spark/checkpoints/*
chmod -R 777 ~/RetailHub-Spark/checkpoints

3. Fix Airflow DAG and Log Folder Permissions
sudo chown -R 50000:0 ~/RetailHub-Spark/airflow/dags ~/RetailHub-Spark/airflow/logs
sudo chmod -R 775 ~/RetailHub-Spark/airflow/dags ~/RetailHub-Spark/airflow/logs
```

---

## 5. Terraform Deployment & Teardown Commands

```
Navigate to Terraform Directory
cd ~/RetailHub-Spark/terraform

Copy Configuration Variables Template
cp terraform.tfvars.example terraform.tfvars

Initialize Providers
terraform init

Review Plan
terraform plan

Apply & Deploy to AWS
terraform apply -auto-approve

View Public ALB URL & Outputs
terraform output

Connect to any EC2 securely via AWS Systems Manager
aws ssm start-session --target <instance_id>

Clean Teardown (Delete everything to prevent charges)
terraform destroy -auto-approve
```

---

\*End of Documentation ? RetailHub Engineering Handbook (2026).\*